

A framework for adapting agile development methodologies

Lan Cao¹,
Kannan Mohan²,
Peng Xu³ and
Balasubramaniam Ramesh⁴

¹Department of Information Technology and Decision Sciences, College of Business and Public Administration, Old Dominion University Norfolk, VA, U.S.A.; ²Department of Statistics and Computer Information Systems, Zicklin School of Business, Baruch College, City University of New York, New York, NY, U.S.A.; ³Department of Management Science and Information Systems, College of Management, University of Massachusetts Boston, Boston, MA, U.S.A.; ⁴Board of Advisors Professor of Computer Information Systems, J. Mack Robinson College of Business, Georgia State University, Atlanta, GA, U.S.A.

Correspondence:

Balasubramaniam Ramesh, Board of Advisors Professor of Computer Information Systems, J. Mack Robinson College of Business, Georgia State University, Suite 903, 35 Broad Street, Atlanta GA 30303, U.S.A.
Tel: +1 404 413 7372;
Fax: +1 404 413 7394;
E-mail: bramesh@gsu.edu

Abstract

Agile development methodologies such as Extreme Programming are becoming increasingly popular due to their focus on managing time to market constraints and the ability to accommodate changes during the software development life cycle. However, such methodologies need to be adapted to suit the needs of different contexts. Past literature has paid little attention to examine the adaptation of agile methodologies. Using adaptive structuration theory as a lens to analyze data from a multisite case study, we examine how the structure of agile methods, projects, and organizations affect the adaptation of agile methodologies. We describe the various sources of structure that affect appropriation of agile practices, the set of appropriated practices and their characteristics, and their link to process outcomes. Based on our findings, we provide prescriptions for adapting agile development methodologies. We also discuss how adapted agile practices can address several challenges faced by agile development teams.

European Journal of Information Systems (2009) 18, 332–343.

doi:10.1057/ejis.2009.26; published online 18 August 2009

Keywords: agile development; process adaptation; adaptive structuration theory; process tailoring; Extreme Programming (XP)

Introduction

The goal of agile methods is to help software development organizations to quickly develop and change their products and services thereby providing the ability to adapt to dynamic market conditions (Highsmith & Cockburn, 2001; Boehm, 2002). Agile methods such as Extreme Programming (XP) advocate iterative development, frequent consultation with the customer, small and frequent releases, and rigorously tested code (Beck, 2000; Agile-Alliance, 2001; Beck & Fowler, 2001).

Though agile methods are considered suitable only in small-to-medium-sized projects (Highsmith & Cockburn, 2001; Boehm, 2002; Turk & Rumpe, 2005), software development organizations are increasingly recognizing the need for agility in almost every project they execute (Lyytinen & Rose, 2006; Mathiassen & Pries-Heje, 2006) due to challenges resulting from a changing environment, ambiguous user requirements, and schedule pressure (Boehm, 2002; Maurer & Melnik, 2006; Abrahamsson & Still, 2007). However, the use of agile methods also poses challenges, such as the lack of sufficient architecture planning, overemphasis on early results, and low levels of test coverage (Highsmith & Cockburn, 2001; Boehm, 2002). Efforts have been made to apply agile methods in large, complex projects to achieve faster development cycle times, but with mixed results (Elshamy and Elssamadisy, 2006; Reifer, 2003). Some critics argue that agile methods cannot be adopted in large, complex projects (Rasmussen, 2003). Though the importance of adapting agile methods is well recognized, our current

Received: 5 May 2008
Revised: 30 September 2008
2nd Revision: 10 February 2009
Accepted: 2 July 2009

understanding of the process of their adoption is limited (Abrahamsson *et al.*, 2003; Fitzgerald *et al.*, 2006). In this research, we define adaptation as the process of changing agile methods to align them with the needs of different projects and organizational environments. This process involves adding, dropping, or modifying specific practices prescribed by agile methods.

Our research investigates the following question: *How are agile software development methodologies adapted for use in different contexts?* To answer this research question, we draw from adaptive structuration theory (AST) that examines organizational change facilitated by different types of structures provided by advanced technologies, tasks, and organizational environments, as well as structures that actually emerge in social action. In our context, agile methods provide a structure for software projects and an adapted structure emerges due to social action of the participants in the process. AST provides us a suitable lens to understand the emergent structure that results from using agile methods in different contexts. We conducted a multisite case study to examine the adaptation of agile practices and develop a framework guided by AST that describes the emergent structure provided by the adapted agile practices.

The remainder of the article proceeds as follows. First, it reviews the various challenges faced in adopting agile methods in different contexts and draws from AST to examine how agile methods are adapted to address these challenges. It then discusses the cases and research methodology used. The next section reports the findings of our study. The article concludes with contributions to theory and practice, and suggestions for future work.

Theoretical background

Challenges to adopting agile methods

The goal of adaptation is to address the challenges associated with the application of agile practices in different projects and organizations. We discuss the main challenges that motivate the adaptation of agile practices in the following sections.

Development process-related challenges

Agile methods value a working system over documents (Agile-Alliance, 2001). No formal architecture design is included in a typical agile approach. Agile methods such as XP advocate continuously refactoring code to improve code quality and system architecture (Beck, 2000). However, lack of architectural scalability can raise a serious concern for relatively large projects. Without a formal design phase, many design problems may be ignored (Erickson *et al.*, 2005). For example, agile development teams may make irrecoverable architectural mistakes due to inadequate attention paid to architectural design (Boehm, 2002).

Customer-related challenges

Agile methods such as XP rely on inputs from on-site customers instead of predefined requirements documents (Beck, 2000). The focus is not how to minimize changes in a project but how to better handle inevitable changes throughout the life cycle (Highsmith & Cockburn, 2001). Agile methods respond to this expectation by adopting strategies designed to reduce the cost of change throughout a project (Highsmith & Cockburn, 2001). By closely working with on-site customers, the team can obtain immediate feedback and information. However, customers' insufficient knowledge of the requirements due to the complexity and size of the system poses significant challenges (Ramesh *et al.*, 2008). These challenges are even more pronounced when customers are not available or not willing to commit to the project (Fitzgerald *et al.*, 2006).

Developer-related challenges

Agile methods rely heavily on tacit knowledge embodied in development teams. All team members collocate in the same room. Stand-up meetings among team members take place daily. Critical decisions may be left undocumented. There is a lack of formal history of the project for team members to trace and understand the evolution of the system. Communication strategies adopted by agile methods work well for small, highly cohesive teams. However, their use in large, complex projects may result in several challenges. Informal communication may not be effective when dealing with a large number of stakeholders and vast amounts of information (Fitzgerald *et al.*, 2006).

Organization/management-related challenges

Agile methods recommend decentralized decision making. Every team member is informed on the progress of the project and is empowered to make decisions on his/her own. Agile methods work well in organizations that have a flat organizational structure (Boehm & Turner, 2003). However, in organizations that are used to deep hierarchical and centralized decision-making structure, they may conflict with the organizational culture, causing resistance between top management and team members (Boehm & Turner, 2003).

Adaptation of agile methods for various environments should carefully address the challenges discussed above. In this article, we draw from AST to examine how agile methods are adapted to address these challenges.

Adaptive structuration theory

AST, which highlights the use of structuration theory in information system (Jones & Karsten, 2008), examines how people incorporate information technologies into their work processes (DeSanctis & Poole, 1994). It focuses on social structures defined by rules and constraints provided by technologies. Although AST links social structure primarily to the structure provided by advanced information technologies, structure can also be provided

by processes and organizations (DeSanctis & Poole, 1994). We use the term *artifacts* to refer to all sources of structure. Artifacts embody a collection of structures that are then instantiated in social environments. Thus, structures present themselves in two forms – embedded in artifacts (technologies, processes, etc.), and in social action. Past research has argued that in order to understand organizational change, we need to gain insights into the interaction between these two forms of structure – viz., structures in artifacts and in action (DeSanctis & Poole, 1994). Further, social structures are described in terms of their structural features and the spirit of these features that may be open to competing interpretations. Although structural features are represented by the rules, constraints, and capabilities of the artifact, spirit refers to the general intent of the artifact's features. The artifact itself, the task that uses the artifact and the organizational environment can be potential sources of structure. In essence, AST highlights *structuration*, which refers to the act of bringing rules, resources, and other structures into action, and *appropriation*, which refers to the application of the structure in a particular context. Appropriation creates 'structures in use' when groups select and interpret specific structures from the 'structural potential' afforded to them by the artifact in consideration (Poole & DeSanctis, 1990). Poole & DeSanctis (1990) argue that as structures are used, they are produced and reproduced resulting in emergent forms of structure. Four aspects of appropriations have been identified by past research – appropriation moves, faithfulness of appropriation, instrumental uses, and attitude (Sambamurthy 1989; DeSanctis *et al.*, 1991; Zigurs *et al.*, 1991). Appropriation moves include the following: (1) Direct use of structure provided by the artifact; (2) Blending with other structures; (3) Constraining the structure through interpretation or reinterpretation; and (4) Expressing judgments about the structure. Faithfulness of appropriation refers to the extent to which structures in use are consistent or faithful to their spirit. Instrumental use refers to the purpose of appropriation. Finally, attitude refers to the perceptions and feelings of users toward the artifact (such as their confidence in appropriation and their perception of usefulness).

AST has primarily been applied to examine the adaptation of advanced information technologies such as group decision support systems that enable organizational change (Dennis & Garfield, 2003; Limayem *et al.*, 2006). It has also been used to examine the dynamics of global virtual teams (Maznevski & Chudoba, 2000). In this study, we use AST as our theoretical foundation to study how agile methods can be adapted and appropriated in different contexts. We argue that the following constitute sources of structure that influence agile method appropriation: (1) agile methods defined through their structural features and spirit; (2) software project characteristics; (3) organizational context; and (4) each team's internal system that includes interacting

style, knowledge, and experience with agile methods, and perceptions about agile methods.

Research methodology

We conducted a multisite case study of four projects in three organizations that had adopted XP. Case study method is best suited to develop an understanding of the interactions among information technology innovations and organizational context (Yin, 1989; Seaman, 1999). It is a suitable method for our study because we seek to examine the phenomenon of agile methods adaptation in its natural settings (Benbasat *et al.*, 1987). Multiple case designs are appropriate for this study since our research objective is to investigate how to adapt agile methods to suit to various contexts. Multiple case designs enable generalizability and the ability to extend theory through cross-case analyses (Miles & Huberman, 1994).

Overview of study sites

Our research question guided the choice of the case study sites. Using theoretical sampling (Patton, 1990), we selected four projects that adapted agile methods. Each case presents a unique context for the adaptation of agile methods. We include both positive experiences and challenges in adaptation to highlight the various facets of adaptation. The use of four varied cases drawn from different contexts (in terms of industry, project size, duration, application type, etc.) strengthens the generalizability of our findings. Table 1 provides relevant details about the four projects studied. It also lists the agile practices that were used in the projects (either as prescribed or adapted to suit their contexts) at various points in the development life cycle.

FinanApp is a complex enterprise system that is used by banks, insurance companies, and brokerages for providing financial services. The application requires high level of reliability and security, which in turn requires appropriate definition and enforcement of its processes. In the first stage of the project, there was a growing recognition among the team members that lack of agility made it difficult to accommodate evolving requirements. The project team decided to adopt essential aspects of XP to achieve agility and improve communication. For example, planning game was used to prioritize the stories for each iteration that lasted 2–4 weeks. Pair programming was adopted for analysis, design, and test case development. The product manager (as a surrogate customer) met with the development team frequently to discuss changes in requirements and to make decisions on development options. The project team had successfully delivered several applications using agile approach. However, in the second stage of the project, when BankSys was acquired by an organization that values formal documentation and procedures, FinanApp had to adapt its practices to address challenges in the new organizational environment.

EventApp is a subsystem in a large suite of network security software. The project team was very experienced

Table 1 Summary of the cases

Organization	BankSys	SecureSys		Transport
Project	FinanApp	EventApp	ProtectApp	TransApp
Size	1000+classes	434 classes About 4600 lines of code	NA	30,000 lines of code
Project Duration	3+years (on going at the time of our study)	6 months	2+years (on going)	8 months
Team	22 developers	Started with 8 persons and increased to 12 persons	50+developers, 5 managers	12–15 developers
Application type and domain	Enterprise financial system	Internal application for network security system	Enterprise security system	Internal application for logistics
Informants and interviews	Senior architect, developer	Technical lead, senior developers (3), quality assurance, product manager (surrogate customer)		CIO, senior manager, project manager, architect, senior developer, web developer
Motivation	Improve agility to accommodate changes; improve communication	Introduce all the XP practices in the organization		Improve responsiveness to changing customer requirements
Agile practices adopted and/or adapted	Planning game, small releases, short iteration, coding standard, refactoring, collective ownership, continuous integration, and customer involvement	Planning game, small releases, short iteration, coding standard, test driven development, refactoring, pair programming, collective ownership, continuous integration, and customer involvement	Small releases, short iteration, continuous integration, and customer involvement	Planning game, small releases, short iteration, coding standard, test driven development, refactoring, paired development activities, collective ownership, continuous integration, and customer involvement
Outcomes	The first stage of FinanApp was considered successful. Applications were delivered within budget and on schedule. During the second stage, FinanApp faced applications on schedule.	EventApp was considered successful. It was completed within budget, and on schedule. The system was evaluated to be of high quality and the customer satisfaction with the project was high.	The final release of ProtectApp was considered not 'so great' by the product manager, when compared with EventApp.	Project was considered successful. Application was delivered within budget and schedule. Customer was very satisfied with functionality and quality.

in the business domain and had successfully piloted some of the XP practices in another project. The team decided to adopt all the XP practices to fully exploit the benefits of the approach. The system was successfully delivered in 6 months and the project manager believed that the team delivered twice as much output as what it would have delivered if it had used a traditional approach. The product manager who acted as the customer liaison was also satisfied with the quality of the delivered product.

After EventApp was completed, the whole team moved to ProtectApp, an enterprise security system. ProtectApp is the core product of the organization with a three-tiered, complex architecture. The 12 members who moved to ProtectApp from EventApp did not secure the same level of support from the managers and other team members for using the full set of XP practices. Although they used practices such as short iterations, small releases, and continuous integration, some practices such as pair programming and unit testing were dropped. According to the product manager, the quality of ProtectApp was not as high as that of EventApp.

TransApp is an application for monitoring the progress of components in a supply chain. The system was first

developed for internal use and later rolled out to external customers. Given the complexity of the domain and the criticality of the process to the business, the customer required stringent quality. However, since this application was being developed in a relatively new domain, the customer requirements were not well specified at the outset. Due to the fluidity of requirements, the development team adopted an agile development approach. However, the strict quality guidelines that were prescribed for TransApp to interface with corporate databases necessitated the adaptation of some of the agile practices used in the project.

Data collection

Qualitative data were collected through semistructured interviews. Data collection and analysis were conducted in an iterative fashion. The interviews focused on gaining insights into the development processes followed in the four projects (an excerpt of the interview guide used in the study is shown in Table 2). Each interview spanned about 2 hours on average. The interviews were recorded and transcribed. Further, technical reports developed as a part of these projects and presentations made by project

Table 2 Excerpt from the semistructured interview guide

1. Please tell us about yourself, your work here, and job title.
2. How many years have you been working in software industry?
3. Please describe the project that adopted agile methods. What were your roles in this project?
4. Please describe the agile practices that were adopted in this project.
5. Please tell us how the team adapted these agile practices for the project.
6. What were the main challenges in adopting agile practices?
7. Please tell us how the decisions on adaptation were made and why such decisions were made.
8. What were the major concerns or goals when adapting these agile practices?
9. During the adapting process, what factors did the team consider?
10. How do you think these factors affect your decisions on adopting and adapting agile practices?

personnel were also used as additional data sources. Follow-up interviews were conducted with informants, and secondary sources were carefully reviewed to corroborate observations and triangulate our findings (Eisenhardt, 1989; Miles & Huberman, 1994).

Data analysis

Coding procedures suggested by Strauss & Corbin (1990) were followed to analyze the qualitative data. We first conducted within-case analysis for each of the cases and then did cross-case analysis to compare the cases (Miles & Huberman, 1994). In the first phase, events and practices related to how adaptation happened, why it happened, what processes were followed, source of influence, and the impact on the teams and organizations were identified and labeled. The AST framework provides guidance on the initial coding scheme. Initially, four researchers separately analyzed the qualitative data and labeled the concepts that represent the source of structure of agile methods such as task characteristics and the spirits of agile methods, organizational factors such as culture, appropriation process and practices such as forward refactoring and empowerment through shared expertise, process outcomes such as customer-related and developer-related processes, project outcomes, and new social structure as the result of adopting agile methods. Then the labels were debated and modified until the researchers reached agreement. The concepts were validated in the subsequent interviews and/or with secondary data. Ambiguous concepts were clarified in follow-up interviews. After several iterations of initial coding, the labels identified in the first stage were then grouped into appropriate categories guided by AST (Figure 1). Finally, the relationships among these theoretical concepts were established to organize our findings. The concepts identified in our analysis corresponding to sources of structure, appropriated practices, and characteristics of appropriation are described in the next section.

Findings

Sources of structure

XP as a source of structure: Tenets of XP provided a source of structure in the process of defining adapted practices. The structural features within XP include practices such as pair programming, iterative development, limited upfront design, unit testing, and continuous integration. The spirit of XP lies in its values such as communication, quick response to change, customer collaboration, and delivering value (Agile-Alliance, 2001).

Other sources of structure: The successful use of agile practices is contingent on task and organizational characteristics. For small-to-medium-sized projects, such as EventApp, agile practices are employed as they are defined. However, for large, complex, and security-critical projects, such as FinanApp, ProtectApp, and TransApp, these practices need to be tailored in order for them to be effectively applied. Also, the change in organizational culture (e.g., the acquisition of BankSys by another company) and changes in the attitudes of top management (in ProtectApp) also influenced the adaptation of some XP practices.

Team's internal system: Team members' knowledge and experience with XP played a role in appropriation of agile practices. All the members in EventApp had used XP in previous projects and were committed to using XP in this project. Similarly, the whole team in FinanApp was encouraged to learn about this relatively new method. TransApp was given the flexibility to choose XP practices that were considered appropriate for the project. In contrast, some managers and developers in ProtectApp team did not believe in XP and were reluctant to try the agile practices.

Structures provided by these various sources impact both the appropriated practices and the characteristics of appropriation discussed in the next two sections.

Appropriated practices

The adaptations of agile practices in the four projects can be categorized into four groups. Each of these groups of practices addresses the corresponding challenges presented earlier.

Development process related

Abstraction in architectural design: For a complex system, such as FinanApp, it is challenging to directly adopt the agile practice of 'simple design' because the system had to be adapted for multiple clients. Therefore, FinanApp 'first designed a completely abstract system that could apply to banking, to insurance, to loans and brokerages'. Such an approach necessitated a robust, reliable architecture that separated the backbone from domain-specific features. It also developed numerous patterns that could be used to quickly identify the appropriate design strategies. ('Each time we get a new service, I design the data model ... what I am doing is mapping the pattern ...'). Similarly, TransApp had to adhere to the very strict guidelines on

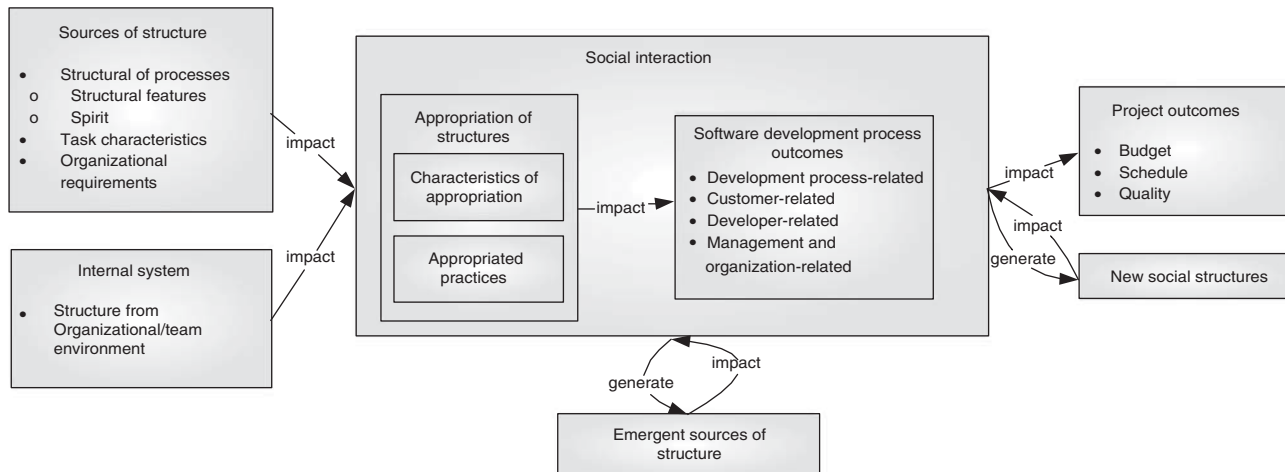


Figure 1 AST adapted for the context of appropriation of agile methods.

the architecture that was prescribed by the corporate information technology department. A three-tier architecture standard was mandated for all the projects to 'save precious time' involved in integrating and maintaining different applications.

Forward refactoring: Recognizing the need to maintain the agile spirit in their methodology, FinanApp adapted 'refactoring' as a mechanism for reusing existing components. Through 'forward refactoring', the team created and validated reusable components for multiple projects. Although the team realized that 'planned reuse is not within the spirit of the agile approach followed', they used forward refactoring as a mechanism to abstract out functionalities.

Design by formalized agreement: Agile methods propose using 'on-site customers' to replace heavy documents (such as detailed requirements specifications) with 'just enough documentation'. However, these practices face challenges in large, complex systems such as FinanApp. Limited formalization was introduced in the requirements management process by seeking business analyst and customer approvals after intense collaborative development of requirements. ('So when the developers were writing the test cases, they turned back over to business analysts and said this is what we were implementing based on your expectation. Business analysts signed off on it. It became a contract'.) TransApp, the other large, complex system adopted a similar approach. It typically eschewed formal specifications in favor of collaborative development. However, after intense collaboration, a consensus was reached and both the customer and the development team signed off on the 'requirements that both parties agreed upon as the basis for development'.

Minimal traceability: FinanApp adapted the agile practice of 'just enough documentation' and 'on-site customer' by incorporating minimal traceability. It established limited traceability between requirements and other artifacts developed so that the development team

could answer questions such as 'did we implement all the functions we intended to implement?' that were often raised by management. At EventApp, the intense involvement of the product manager in each iteration ensured that customers' needs were traced to the implementations. Similarly at TransApp, minimal traceability between system components and these requirements was maintained so that the development team could verify that the implementation 'met all the requirements'.

Post hoc documentation: FinanApp modified the agile practice of 'planning game', which defines how requirements are gathered and implemented. It created minimal documentation after the changes were implemented in the system so that documentation did not get in the way of implementation. ('The system has to evolve, has to be validated, and then you can do documentation to describe what you have'.) Although upfront documentation was considered unnecessary, the development team at TransApp also felt the need to 'freeze requirements' after a few 'rounds of experimentation and negotiation especially because of the churn in requirements'. It also adopted *post hoc* documentation of requirements after they became stable. Such 'after the fact' documentation, while allowing the development team to achieve agility, also satisfied organizational norms. This somewhat unusual practice 'helped speed up the process dramatically rather than getting bogged down by formal upfront documentation'.

Minimal documentation: The strict organizational requirements tempered with the agile spirit resulted in TransApp creating 'minimal, but necessary documentation'. The team negotiated with the software process group and identified a few documents that were necessary for 'verifying that the implementation meets the necessary quality requirements'. In contrast, while FinanApp initially adopted the agile practice of 'just enough documentation', after the acquisition, senior management who did not have a favorable attitude toward agile

methods viewed detailed documentation as necessary. ('...they [the senior management] still think that waterfall model and documentation can completely and correctly describe the system'.)

Developer related

Pairing for overlaps: 'Pair programming' was adapted to suit the developers' style and the nature of the tasks performed. EventApp paired developers in all tasks to enhance knowledge sharing and training. In FinanApp, only analysis, design, and test case development were done in pairs. Most of the developers who tried to write code in pairs returned to working alone. TransApp experimented with paired development in the beginning, but quickly found that it 'was creating tensions that were not conducive to the development goals'. The team decided to use pairing only in select aspects of the process so that individuals could get their 'own space when they wanted it'. The team used pairing extensively in 'reviews and high-level design'.

Empowerment through shared expertise: The spirit of 'collective ownership' was encouraged by motivating developers to be involved in all aspects of the project. Both FinanApp and EventApp encouraged their team members to co-own the project. Team members were rotated to gain familiarity with different aspects of the project. ('Basically the approach we took was [that] everybody wanted to learn how to do everything. Everybody touched everything'.) EventApp emphasized the need for all developers to acquire knowledge about both the domain and the technology. The team learned the business value of each feature through the interaction with the product manager. EventApp created a collaborative environment in which the team worked in an open lab. ('It was helpful because I might be working on one task on one pair station, and the station right next to me the guy was talking about an issue they are trying to solve. Next thing I know they are talking about something that [will be] impacting my area'.) At TransApp, the entire team felt empowered and team members had 'high degree of autonomy in choosing the tasks [they] wanted to be involved in at any [part of the] development cycle'. This helped any team member 'try [his/her] hand at various aspects of the project'.

Customer related

Shared understanding of specifications: TransApp, FinanApp, and EventApp adapted the agile practice of 'on-site customer' by using product managers as surrogate customers when actual customers were not readily available. At TransApp, the intense involvement of the surrogate customer in the development process helped develop a 'common understanding of the expectations of both the customer and the development team members'. This ensured that all the stakeholders were 'on the same page'. In both FinanApp and EventApp, the product manager discussed each feature with the team before it

was implemented to enhance the 'understanding of the task and the business reasons'.

Agreement on quality: FinanApp recognized that different stakeholders may have different notions of quality and adopted the practice of 'planning game' that facilitated intense communication that was necessary to reach an agreement on quality. ('My definition of quality is that the delivered product does roughly what the customer expected, and these people could agree, and documentation supports them. But the most important thing is that BA, QA, and developers all have to be in line'.) At TransApp, the software process group that represented corporate standards insisted that all projects 'arrive at a consensus on quality requirements' that satisfied all the stakeholders.

Organization/management related

Upfront estimation: Although the initial experimentation with agile development was considered successful, the senior management at TransApp insisted that future projects must have clearly defined costs and benefits. This necessitated the adaptation of the practice of 'planning game' to an upfront estimation approach in which strong assumptions were made about project scope, in contrast to typical agile development where the scope is expected to change (often considerably) during development. However, the management was willing to 're-evaluate project costs and benefits if there were significant changes' during development.

Balanced formality: TransApp followed a process that was viewed by the senior management as somewhat 'radical and informal', and, therefore, they required the team to incorporate some level of formality in the process so that the outputs of the agile process would be acceptable to the rest of the organization, especially the corporate information technology organization that needed to 'bless the output' for organization-wide use. Thus, a hybrid process that satisfied both groups emerged. ProtectApp found it challenging to balance the formality demanded by the different team members and the management because of the misalignment between their informal and formal cultures.

In summary, a variety of practices were observed in the three organizations adapting agile processes. These practices provide prescriptions for software development organizations to adapt agile processes for different contexts. The following section describes how these practices were adapted and implemented by these organizations.

Characteristics of appropriation

Here, we describe how appropriation methods and other sources of structure are brought into action. AST suggests four characteristics: appropriation moves, faithfulness of appropriation, instrumental uses, and attitude toward appropriation.

Appropriation moves As a medium-sized project, EventApp directly adopted most of the proposed practices in

XP. However, when the same team members moved to a large project, ProtectApp, with a new director, managers, and additional team members, the team adopted only a subset of XP practices.

FinanApp's appropriation of XP was more complex due to the size and complexity of the project. Initially, FinanApp adapted many XP practices to manage challenges caused by the size and complexity of the project. Some XP practices, such as pair programming, were modified into 'pairing for overlaps'. However, after FinanApp was acquired by another company, the new management '[was] extremely heavy' and believed in traditional procedures. The sudden shift in management style and culture caused some conflicts and re-shaped the appropriated practices that once worked well, making the team modify practices to suit the needs of the new management.

TransApp adapted XP practices to conform to strict quality guidelines of the organization. Because of the prevailing organizational culture, the strict use of some XP practices was impractical. For example, they realized that forcing pair programming created tensions. Therefore, they paired for only selected aspects of the process such as review and high-level design.

Faithfulness Although EvenApp adopted XP fully, the other three projects adapted XP practices at different levels. The faithfulness of appropriation is concerned with whether the appropriation is consistent with the agile spirit (DeSanctis & Poole, 1994). FinanApp's appropriation involved two stages. The first stage involved faithful appropriation of XP. Though some traditional practices were blended with agile, the FinanApp team tried to establish the spirit of agility, while balancing agility with scalability. They adopted many agile practices that promoted the agile spirit. During the second stage, the FinanApp team faced considerable difficulty in reconciling its view of agility with top management's view of implementing software processes. For example, although agility demanded increased emphasis on creativity in programming, after the acquisition, the new management at FinanApp imposed stringent processes that treated programming as a 'brain dead activity'. Similarly, differences in the vocabulary and metaphors used by new top management and the development team at FinanApp resulted in misunderstanding and mistrust. For instance, 'refactoring' carried different connotations for top management and developers with agile mindset. Similarly at ProtectApp, although some XP practices were adopted, due to the lack of support from management and peers, the use of agile practices was less faithful to the spirit of agile. For example, continuous integration was used for revision control instead of supporting the quick pace of development. Although TransApp followed most of the XP practices initially, the senior management later recognized the need to incorporate some level of formality to conform to organizational standards.

Appropriation in TransApp was still consistent with the spirit of XP – bringing agility to the development process.

Instrumental uses At EventApp, the purpose of appropriating XP was to benefit from this methodology. EventApp piloted some XP practices before starting the project and realized that agile practices support each other. Then the team decided to appropriate all the XP practices to fully benefit from the approach. Both FinanApp and TransApp appropriated parts of XP to bring agility to the large project to enable responsiveness.

Attitude At EventApp, the team had gained experience and knowledge about key XP practices in a pilot project. Team members with an 'agile mindset' were hand picked for the project. They were committed to XP and had a very positive attitude toward its direct and faithful appropriation. When these team members moved to the ProtectApp project, though they were positively disposed toward XP, others in the project were not. At FinanApp, the team members were committed to the modified XP practices before new management took over. Later, though the team still tried to continue its appropriation of XP, new management showed a hostile attitude toward the appropriation. At TransApp, both the development team and senior management were committed to XP with a positive attitude toward appropriation.

Characteristics of appropriation discussed here help software development organizations better understand the precursors of process outcomes, thereby enabling them to develop prescriptions for process adaptation.

Discussion

Our findings are summarized in the framework shown in Figure 1. Agile methods provide structure through the rules and constraints they advocate. Project and task characteristics also impose structure on the development team. Organizational requirements and practices that are considered as norms in the current organizational environment also constitute sources of structure. Moreover, the team's internal system including team members' experience with and attitude toward the development approach is also a source of structure.

DeSanctis & Poole (1994) include the characteristics of appropriation (which includes appropriation moves, faithfulness, instrumental uses, and attitude) in their AST model. We extend the AST model by expanding the 'appropriation' construct to include characteristics of appropriation and detailed appropriated practices when adapting agile methods. In our context (of software development process adaptation), appropriation impacts software development process outcomes (i.e., development process-related, customer-related, developer-related, and organization/management-related outcomes) that address the challenges that we discussed earlier.

Table 3 summarizes how these challenges are addressed by appropriated practices followed in the focal

Table 3 Appropriate practices and challenges addressed

Category	Appropriated practices	Agile practices that were adapted	Addressing challenges by adapting agile practices
Development process related	Abstraction in architectural design	Simple design	These appropriated practices address development process-related challenges by focusing on balancing design principles that have been hallmarks of agile methods with discipline They inject required formality and planning into agile practices
	Forward refactoring	Refactoring	
	Design by formalized agreement	On-site customer Just enough documentation	
	Minimal traceability	On-site customer Just enough documentation	
Developer related	<i>Post hoc</i> documentation	Planning game	These appropriated practices address developer-related challenges by building a cohesive and empowered team, and emphasizing developers' autonomy and shared ownership
	Minimal documentation	Just enough documentation	
	Pairing for overlaps	Pair programming	
	Empowerment through shared expertise	Collective ownership	
Customer related	Shared understanding of specifications	On-site customer	These appropriated practices address customer-related challenges by gaining customer involvement required in an agile approach
	Agreement on quality	Planning game	
Organization/management related	Upfront estimation	Planning game	These appropriated practices address organization/management-related challenges by striving to achieve better alignment between organizational culture and agile practices
	Balanced formality	Informal processes	

organizations of our case study and the agile practices from which they were adapted.

Past research on software development processes focuses largely on understanding the suitability of a particular type of process for different contexts, aligning processes with organizational contingencies, examining how different processes can be complementarily integrated to be effective in the face of volatile requirements, and examining the effectiveness of specific tenets of various methodologies. However, the larger question of gaining insights into why processes result in either remarkable or inadequate improvement in project outcomes requires understanding of how various processes are appropriated. AST provides a suitable framework to examine this question. In this research, we have taken the initial step in examining the adaptation of processes. This study tries to understand the interplay between software development processes and the social process of implementing these processes in different contexts. It advances the view that processes and their tenets show the structural potential that is utilized in appropriation through social interaction rather than presenting a set of

immutable rules and constraints to be adopted without friction. We highlight how such structural potential (provided through structural features of the processes and their spirit) are appropriated.

Examination of the appropriation practices in our case study reveals that while several appropriations were effective and consistent with the spirit of agile development methods, a few additional challenges in appropriation emerged. For example, the principle 'people over process', a central aspect of agile methods, sometimes is inconsistent with top management's view of programming as routine, which is rooted in their view of traditional process and their cultural norms. This resulted in irreconcilable differences without an effective appropriated practice. The use of AST facilitates such scrutiny of process-culture misalignments and emphasizes on understanding the appropriation process to gain insights into reasons for such results in social interaction.

One important aspect highlighted in our study of process adaptation is the 'spirit' of a process in addition to the structural features of the process that constitutes the structural potential. Process implementations that are

not aligned with the spirit or general intent of the process are likely to result in adverse project outcomes when compared to those that are faithful to the spirit. When examining spirit, we focus on the goals that are promoted by the process rather than implementing the tenets of the process rigorously. For instance, the spirit behind practices such as pair programming should be examined when implementing them in different project contexts. Our study also shows that the faithful appropriation of agile methods leads to more project success. EventApp has higher product quality and customer satisfaction than ProtectApp. Similarly, FinanApp was more successful before it was acquired. However, it should be noted that faithfulness of appropriation also interacts with context in shaping process outcomes. For instance, a faithful appropriation of agile practices in a context that is not conducive to the implementation of agile practices is not likely to be successful.

Another important element of AST that demands attention in the context of process adaptation is the emergence of new sources of structure as the processes are applied in social action. For instance, when specific practices are implemented in a particular team, patterns of good implementation of these practices might emerge along with data about the implementation of these practices. These patterns and data may reproduce themselves as new rules and constraints that affect further implementation of these practices. In essence, appropriation of specific structural features from existing processes might generate additional sources of structure, which in turn will impact the appropriation process. This results in a cycle between structure and appropriation. Also, structures in action are produced and reproduced over time so they result in new structures that constitute rules and constraints in implementing specific software development practices. These new structures may map directly to the adapted practices that we discussed earlier that consider not only existing process structures but also organizational and project characteristics. These adapted practices implemented over time become new sources of structure.

It should be noted that while we adapt AST to the context of process adaptation, the primary focus of our article has been on the appropriation process. Gaining additional insights into emergent sources of structure, new social structures, and the link between social interaction and project outcomes (such as budget, schedule, and quality) are subjects of ongoing research.

Extending AST theory and practice

Our study contributes to theory in the following ways:

1. *Grounding process adaptation and agile methods in theory:* Agile methods and the area of process adaptation in software development have been largely developed based on practice, with limited application of theory. AST provides a suitable theoretical lens to examine process adaptations as appropriated practices in social interaction of teams.
2. *Extending AST:* Although the AST model included sources of structures and appropriation as constructs, we extend these constructs in our research. Four characteristics of the appropriation construct in AST are appropriation moves, faithfulness, instrumental uses, and attitude. We expand the appropriation construct in our framework highlighting the difference between characteristics of appropriation and appropriated practices.
3. *AST in a novel context:* AST has been primarily applied in the context of organizational change induced by advanced information technologies. By adapting AST with 'software development process' as the focal artifact instead of advanced information technology, it becomes clear that processes provide rules and constraints through their structural tenets and spirit. This is similar to DeSanctis and Poole's argument that advanced information technology provides structure through its features and spirit. These structures are appropriated by development teams in the process of social action. We document the nature of appropriation of such structures and detail appropriated practices in the context of adapting agile development methods.

Our study contributes to practice in the following ways:

The guidelines for adapting agile practices proposed in this research can help the following stakeholders involved in software development in adapting agile development methodologies for different contexts:

1. *Top management* needs to recognize the importance of their role in the extent to which agile or adapted agile methods are adopted. Our findings indicate that the successful adoption of agile practices requires top management support. For example, agile practices that were successfully adopted at FinanApp faced significant challenges after new management, which was hostile to agile methods, took over the company. Similarly, ProtectApp's top management's conventional view of software development posed considerable obstacles to the adoption of agile practices by the development team. Thus, top management should recognize the need to balance the conventional view of organizational process maturity with the agile methods view based on the nature of the project. Appropriation of agile practices may need to be balanced to address conflicting needs of top management, existing organizational culture, and development team culture. This is especially applicable when different stakeholders are anchored to tenets from various methodologies they have used in the past and refuse to adjust their biases based on demands from current market conditions and development culture in teams.
2. *Project managers* should recognize that tenets of development methodologies cannot be applied without

providing due consideration for their impact on the development team and process-related outcomes. They should recognize the need to temper methods to suit the development styles of teams, project characteristics, and to understand factors that may result in friction with top management. Project managers should recognize the challenges in applying agile methods in their entirety and that tailoring and choice of specific tenets is needed to balance development needs and top management's priorities. Our findings show specific practices that were successfully appropriated and that adaptation of some practices may become misaligned with existing organizational culture. While successful appropriation scenarios presented in our study can inform project managers in examining their current practices and identifying potential opportunities in applying agile practices that may suit their context, the challenges to appropriation reported in our study are equally important. These findings inform project managers to carefully consider scenarios that may not be conducive to adoption of agile methods or for process tailoring.

3. *Developers* need to understand the impact of the significant autonomy provided to them through adapted practices under varying contexts. Many of the practices that we present earlier are prescriptive and concrete. These practices can be applied by developers to handle volatility in requirements without completely sacrificing rigor in the process.

About the authors

Balasubramaniam Ramesh is Board of Advisors Professor of computer information systems at the Georgia State University, and studies requirements engineering and traceability in systems development, agile software development, knowledge management, and web services. His work appears in several leading journals including *MIS Quarterly*, *EJIS*, *IEEE Transactions on Software Engineering*, *JAIS*, among others.

Lan Cao is Assistant Professor of Information Technologies and Decision Sciences at the Old Dominion University. She received her Ph.D. on CIS from the Georgia State University. Her major research interests are agile software development and software process modeling and simulation.

Although we examined the appropriation process across three organizations and four projects, our results may be particularistic to the conditions faced by these organizations. Further large-scale empirical studies should be conducted to examine the applicability of our findings to different types of projects and to identify specific characteristics of projects that should be carefully considered while tailoring development methodologies, and how those characteristics can impact development methodologies. Our extension and application of AST in this context will have to be validated by future studies. Further research is also needed in bringing in theoretically grounded insights into agile development practice.

Concluding remarks

Based on the findings from a multisite case study, we have developed a conceptual framework by adapting AST. Our framework addresses our research question by presenting the various sources of structure that affect the appropriation of agile methods. The framework highlights that agile practices are adapted and appropriated based on the project, organizational, and development context. Our findings include a set of appropriated practices that are shaped by various sources of structures and address several challenges in process adaptation. We also map these practices to software development process outcomes.

Kannan Mohan is an Associate Professor of CIS at Baruch College. Dr. Mohan received his Ph.D. degree in CIS from the Georgia State University. His research interests include managing software product family development, providing traceability support for systems development, knowledge integration, and agile development methodologies.

Peng Xu is Assistant Professor at the Department of Management Science and Information Systems of the University of Massachusetts Boston. She received her Ph.D. in Computer Information Systems from the Georgia State University in 2004. Her research areas are knowledge management, software process, software engineering, and process agility.

References

- ABRAHAMSSON P and STILL J (2007) Agile software development: theoretical and practical outlook. In *Proceedings of the Product Focused Software Process Improvement*, (MÜNCH J and ABRAHAMSSON P, Eds), pp 410–411, Springer Riga, Latvia.
- ABRAHAMSSON P, WARSTA J, SIPONEN MT and RONKAINEN J (2003) New directions on agile methods: a comparative analysis. In *Proceedings of the 25th International Conference on Software Engineering*, (MÜNCH J and ABRAHAMSSON P, Eds), pp 244–254, IEEE Computer Society, Portland, Oregon.
- AGILE-ALLIANCE (2001) Manifesto for Agile Software Development [www document] <http://www.agilealliance.org/>.
- BECK K (2000) *Extreme Programming Explained: Embrace Change*. Addison-Wesley, Boston, MA.
- BECK K and FOWLER M (2001) *Planning Extreme Programming*. Addison-Wesley Longman, New York, NY.
- BENBASAT I, GOLDSTEIN DK and MEAD M (1987) The case research strategy in studies of information systems. *MIS Quarterly* **3**(11), 369–386.

- BOEHM B (2002) Get ready for agile methods, with care. *IEEE Computer* **1**(35), 64–69.
- BOEHM B and TURNER R (2003) Using risk to balance agile and plan-driven methods. *IEEE Computer* **36**(6), 57–66.
- DENNIS AR and GARFIELD MJ (2003) The adoption and use of GSS in project teams: toward more participative processes and outcomes. *MIS Quarterly* **2**(27), 289–323.
- DESANCTIS G and POOLE MS (1994) Capturing the complexity in advanced technology use: adaptive structuration theory. *Organization Science* **2**(5), 121–147.
- DESANCTIS G, POOLE MS, LEWIS H and DESHARNAIS G (1991) Using computing in quality team meetings: initial observations from the IRS–Minnesota project. *Journal of Management Information Systems* **3**(8), 7–26.
- EISENHARDT KM (1989) Building theories from case study research. *Academy of Management Review* **4**(14), 532–550.
- ELSHAMY A and ELSSAMADISY A (2006) Divide after you conquer: an agile software development practice for large projects. In *Proceedings of the XP 2006*, (ABRAHMSSON P, MARCHESI M and SUCCI G, Eds), pp. 164–168, Springer-Verlag, Oulu, Finland.
- ERICKSON J, LYYTINEN K and SIAU K (2005) Agile modeling, agile software development, and extreme programming: the state of research. *Journal of Database Management* **4**(16), 88–99.
- FITZGERALD B, HARTNETT G and CONBOY K (2006) Customising agile methods to software practices at Intel Shannon. *European Journal of Information Systems* **2**(15), 200–213.
- HIGHSMITH J and COCKBURN A (2001) Agile software development: the business of innovation. *IEEE Computer* **9**(34), 120–127.
- JONES MR and KARSTEN H (2008) Giddens's structuration theory and information systems research. *MIS Quarterly* **1**(32), 127–157.
- LIMAYEM M, BANERJEE P and MA L (2006) Impact of GDSS: opening the black box. *Decision Support Systems* **2**(42), 945–957.
- LYYTINEN K and ROSE GM (2006) Information system development agility as organizational learning. *European Journal of Information Systems* **2**(15), 183–199.
- MATHIASSEN L and PRIES-HEJE J (2006) Business agility and diffusion of information technology. *European Journal of Information Systems* **2**(15), 116–119.
- MAURER F and MELNIK G (2006) Agile methods: moving towards the mainstream of the software industry. In *Proceedings of the International Conference on Software Engineering (ICSE)*, (ANDERSON MK, Ed.), pp 1057–1058, ACM, Shanghai, China.
- MAZNEVSKI ML and CHUDоба KM (2000) Bridging space over time: global virtual team dynamics and effectiveness. *Organization Science* **5**(11), 473–492.
- MILES MB and HUBERMAN AM (1994) *Qualitative Data Analysis: An Expanded Sourcebook* 2nd edn, Sage, Thousand Oaks, CA.
- PATTON MP (1990) *Qualitative Evaluation and Research Methods* 2nd edn, Sage, London.
- POOLE MS and DESANCTIS G (1990) Understanding the use of group decision support systems: the theory of adaptive structuration. In *Organizations and Communication Technology* (FULK J and STEINFELD C, Eds), pp 173–193, Sage, Newbury Park, CA.
- RAMESH B, CAO L and BASKERVILLE R (forthcoming) Agile requirements engineering practices and challenges: an empirical study. *Information Systems Journal* in press.
- RASMUSSEN J (2003) Introducing XP into greenfield projects: lessons learned. *IEEE Software* **3**(20), 21–28.
- REIFER DJ (2003) XP and the CMM. *IEEE Software* **3**(20), 14–15.
- SAMBAMURTHY V (1989) Supporting group performance during stakeholder analysis: the effects of alternative computer-based designs. Ph.D. Thesis, University Of Minnesota, MN, U.S.A.
- SEAMAN CB (1999) Qualitative methods in empirical studies of software engineering. *IEEE Transactions on Software Engineering* **4**(25), 557–572.
- STRAUSS A and CORBIN J (1990) *Basics of Qualitative Research: Grounded Theory Procedures and Techniques*. Sage Publications, Newbury Park, CA.
- TURK DRF and RUMPE B (2005) Assumptions underlying agile software-development processes. *Journal of Database Management* **4**(16), 62–87.
- YIN RK (1989) *Case Study Research: Design and Methods*. Sage Publications, Beverly Hills, CA.
- ZIGURS I, DESANCTIS G and BILLINGSLEY J (1991) Adoption patterns and attitudinal development in computer-supported meetings: an exploratory study with SAMM. *Journal of Management Information Systems* **4**(7), 51–70.

Reproduced with permission of the copyright owner. Further reproduction prohibited without permission.